

BMC-301 • PYTHON PROGRAMMING

UNIT I

Introduction to Python, Strings & Control Flow Statements

MCA III Semester

Department of Computer Applications

What We Will Cover in Unit I

01

Introduction to Python

History, applications, program structure, identifiers, keywords, operators, data types, I/O statements

02

Strings

Creating strings, built-in functions, operators, slicing, joining, formatting

03

Control Flow Statements

Conditional statements, loops, nested control flow, break / continue / pass / exit

Learning Outcomes

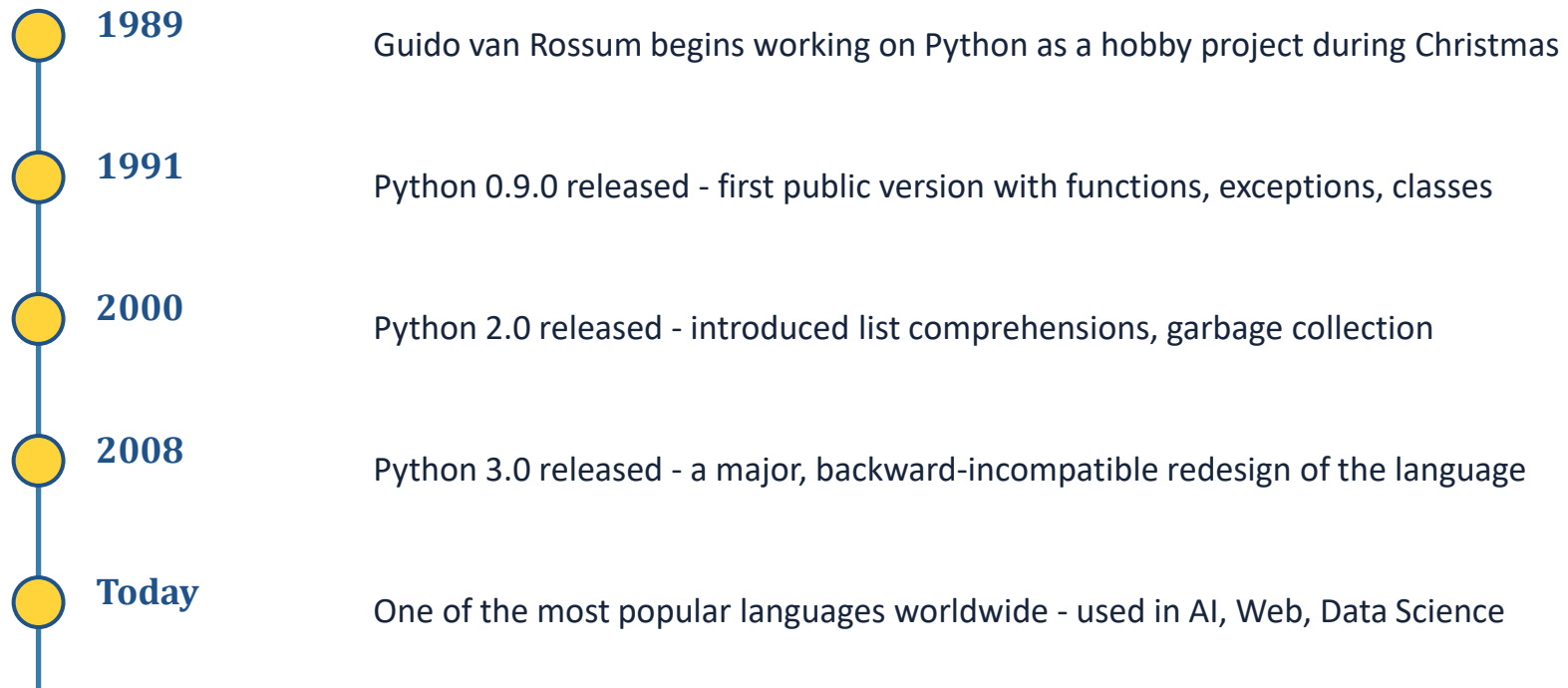
- Understand the origin, philosophy and growth of Python as a programming language
- Identify real-world application areas where Python is widely used
- Write the basic structure of a Python program using correct syntax
- Apply identifiers, keywords, operators and data types correctly
- Use Python's input/output statements for interactive programs
- Manipulate strings using built-in functions, slicing and formatting
- Implement conditional and loop-based control flow in programs

Why This Unit Matters

- Forms the foundation for every program you will write in this course
- Concepts here are reused in Functions, OOP, File Handling (Units II & III)
- Directly maps to Lab Programs 1-9 in BCA-3007P

History of Python

Created by **Guido van Rossum** and first released in **1991** at CWI, Netherlands.



Why Python? Key Features

Language Features

- Simple and easy-to-read syntax
- Interpreted - no separate compilation step
- Dynamically typed - no explicit type declaration
- Free and open source
- Extensive standard library ('batteries included')
- Portable - runs on Windows, Linux, macOS

Programming Support

- Supports procedural, object-oriented and functional styles
- Highly extensible with C/C++ modules
- Huge ecosystem of third-party packages (PyPI)
- Strong community and documentation
- Embeddable in other applications
- Excellent for rapid prototyping

Application Areas of Python

1 Web Development

Django, Flask power large-scale web apps

2 Data Science & AI

NumPy, Pandas, TensorFlow, scikit-learn

3 Automation / Scripting

Task automation, system administration

4 GUI Applications

Desktop apps using Tkinter, PyQt

5 Game Development

2D games using Pygame

6 Scientific Computing

Research, simulations, visualization

Structure of a Python Program

- **Comments**
 - Begin with # — ignored by the interpreter; used for documentation
- **Indentation**
 - Python uses indentation (not braces) to define blocks of code
- **Statements**
 - Each line is normally one statement; semicolon (;) can separate multiple statements
- **Modules**
 - A .py file is itself a module; can import other modules with the import keyword
- **Entry Point**
 - Execution starts from the top of the file - no explicit main() required (though common in practice)

Good Practice

- Use 4 spaces for indentation (PEP-8 standard)
- Never mix tabs and spaces
- Keep one statement per line for readability

Your First Python Program

PY

```
# This is a comment
print("Hello, World!")

# Variables and a simple calculation
name = "BCA Student"
marks = 92

print("Welcome,", name)
print("Your marks:", marks)
```

Explanation

- Comments start with # and are ignored during execution
- print() displays output on the screen
- Variables (name, marks) are created simply by assignment - no type declaration needed
- Multiple values passed to print() are separated by commas and printed with a space between them
- Indentation is not required here since there are no blocks (if/loop/function)

Identifiers

- An identifier is the name given to variables, functions, classes, modules, etc.
- **Rules for naming identifiers:**
 - Must start with a letter (A-Z, a-z) or an underscore (_)
 - Cannot start with a digit
 - Can contain letters, digits and underscores only
 - Cannot be a reserved keyword (e.g. if, for, class)
 - Python is case-sensitive: Marks and marks are different identifiers

Valid vs Invalid

- Valid: student_name, _temp, total1
- Invalid: 1total (starts with digit)
- Invalid: total-marks (hyphen not allowed)
- Invalid: class (reserved keyword)

Keywords in Python

Category	Examples
Value Keywords	True, False, None
Operator Keywords	and, or, not, in, is
Control Flow	if, elif, else, for, while, break, continue, pass
Function/Class	def, return, lambda, class, yield
Exception Handling	try, except, finally, raise, assert
Module Related	import, from, as, global, nonlocal
Other	pass, del, with, async, await

Operators in Python - Overview

- **Arithmetic Operators**

- `+, -, *, /, //` (floor div), `%` (modulus), `**` (exponent)

- **Relational (Comparison) Operators**

- `==, !=, >, <, >=, <=`

- **Logical Operators**

- `and, or, not`

- **Assignment Operators**

- `=, +=, -=, *=, /=, //=, %=, **=`

- **Identity & Membership Operators**

- `is, is not, in, not in`

Arithmetic Operators - Example

PY

```
a = 17
b = 5

print(a + b)    # Addition
print(a - b)    # Subtraction
print(a * b)    # Multiplication
print(a / b)    # Division (float)
print(a // b)   # Floor division
print(a % b)    # Modulus (remainder)
print(a ** b)   # Exponentiation
```

Explanation

- / always returns a float result, even if it divides evenly
- // (floor division) returns the integer quotient, discarding the remainder
- % returns the remainder of division - very useful for checking even/odd numbers
- ** raises the left operand to the power of the right operand
- Output for above: 22, 12, 85, 3.4, 3, 2, 1419857

Operator Precedence (High to Low)

Precedence	Operators	Description
1 (Highest)	()	Parentheses
2	**	Exponentiation
3	*, /, //, %	Multiplication, Division, Modulus
4	+, -	Addition, Subtraction
5	==, !=, >, <, >=, <=	Comparison
6	not	Logical NOT
7	and	Logical AND
8 (Lowest)	or	Logical OR

Precedence in Action

PY

```
result = 10 + 2 * 3 ** 2
print(result)

# Step-by-step evaluation:
# 3 ** 2 -> 9           (exponent first)
# 2 * 9 -> 18          (multiplication next)
# 10 + 18 -> 28        (addition last)

print(10 > 5 and 3 < 7)  # True and True ->
True
```

Explanation

- Python evaluates ****** before ***** and **/**, which it evaluates before **+** and **-**
- Use parentheses to make complex expressions clearer and avoid bugs
- Comparison operators are evaluated before **'and'** / **'or'**
- Final printed value of result is 28

Basic Data Types in Python

int

Whole numbers

```
x = 25
```

float

Decimal numbers

```
y = 3.14
```

str

Text / sequence of characters

```
name = "BCA"
```

bool

True or False values

```
flag = True
```

complex

Numbers with real & imaginary parts

```
z = 2+3j
```

NoneType

Represents absence of a value

```
x = None
```

Type Conversion

PY

```
# Implicit conversion (done by Python)
a = 5          # int
b = 2.5        # float
c = a + b      # int auto-converted to float
print(c, type(c))    # 7.5 <class 'float'>

# Explicit conversion (done by programmer)
x = "100"
y = int(x) + 50
print(y)          # 150

print(str(25))    # '25'
print(float("9")) # 9.0
```

Explanation

- Implicit conversion: Python automatically converts a smaller data type to a larger one to avoid data loss
- Explicit conversion (type casting): programmer uses functions like `int()`, `float()`, `str()` to convert manually
- Converting an invalid string (e.g. `int("abc")`) raises a `ValueError`

Statements and Expressions

- **Expression**
 - A combination of values, variables and operators that evaluates to a single value, e.g. `a + b * 2`
- **Statement**
 - A complete instruction that Python can execute, e.g. an assignment, a `print()` call, an if block
- Every expression can be part of a statement, but not every statement is an expression
- **Examples of statements:**
 - `x = 10` (assignment statement)
 - `print(x)` (function call statement)
 - `if x > 5: print("Big")` (conditional statement)

Input Statement

PY

```
name = input("Enter your name: ")
print("Hello,", name)

# input() always returns a string!
age = input("Enter your age: ")
age = int(age)          # convert to int
explicitly

print("Next year you will be", age + 1)
```

Explanation

- `input()` pauses the program and waits for the user to type something, then presses Enter
- The optional string inside `input()` is shown as a prompt message
- `input()` ALWAYS returns a string - even if the user types a number
- To perform arithmetic, you must explicitly convert it using `int()` or `float()`

Output Statement - print()

PY

```
print("BCA III Semester")
print("Marks:", 92, "Grade:", "A")

# sep and end parameters
print("Python", "is", "fun", sep="-")
print("Loading", end="...")
print("Done")

# f-string for formatted output
marks = 92
print(f"Your marks are {marks}")
```

Explanation

- print() can take multiple comma-separated values, displayed with a default single space between them
- sep changes the separator between values (default is a space)
- end changes what is printed after the statement (default is a newline \n)
- f-strings (prefix f before the quotes) let you embed variables directly using { }

Quick Recap - Introduction to Python

- ✓ Python is a simple, interpreted, dynamically-typed language created by Guido van Rossum (1991)
- ✓ Widely used in Web, AI/Data Science, Automation, GUI and Game Development
- ✓ A program is built from comments, indentation-based blocks and statements
- ✓ Identifiers name variables/functions; certain words are reserved as keywords
- ✓ Operators follow a strict precedence order; data types include int, float, str, bool, etc.
- ✓ `input()` reads a string from the user; `print()` displays output with sep/end control

Coming up in the Lab: We now move on to Strings - the second topic of Unit I.

02

Strings

Creating, manipulating, slicing and formatting text in Python

Creating and Storing Strings

- A string is a sequence of characters enclosed in quotes
- Can be created using single ('), double ("), or triple ('' or ''') quotes
- Triple-quoted strings can span multiple lines
- Strings in Python are immutable - once created, they cannot be changed in place
- Each character has an index: 0, 1, 2, ... from the left, and -1, -2, ... from the right

String Creation - Example

PY

```
s1 = 'Hello'
s2 = "World"
s3 = '''This is a
multi-line string'''

print(s1, s2)
print(s3)

# Strings are immutable
s1[0] = 'h'    # This will raise a TypeError
```

Explanation

- Single and double quotes behave identically in Python
- Triple quotes are commonly used for multi-line text or docstrings
- Attempting to change a character directly (`s1[0] = 'h'`) raises a `TypeError` because strings cannot be modified in place
- To 'change' a string, you must create a new string

String Indexing & Slicing

PY

```
s = "PYTHON"
#   P   Y   T   H   O   N
#   0   1   2   3   4   5
#  -6  -5  -4  -3  -2  -1

print(s[0])      # P
print(s[-1])     # N
print(s[1:4])    # YTH
print(s[:3])     # PYT
print(s[3:])     # HON
print(s[::-1])   # NOHTYP (reversed)
```

Explanation

- Indexing `s[i]` returns a single character at position `i`
- Slicing `s[start:end]` returns characters from start up to (but not including) end
- Omitting start means 'from the beginning'; omitting end means 'till the end'
- A slice with step -1 (`s[::-1]`) is the common idiom to reverse a string

Built-In Functions for Strings

Function	Purpose
<code>len(s)</code>	Returns length (number of characters) of the string
<code>upper()</code> / <code>lower()</code>	Converts string to UPPERCASE / lowercase
<code>strip()</code>	Removes leading and trailing whitespace
<code>find(sub)</code>	Returns the index of first occurrence of sub, or -1
<code>count(sub)</code>	Counts non-overlapping occurrences of sub
<code>replace(old,new)</code>	Returns a new string with old replaced by new
<code>split(sep)</code>	Splits string into a list of substrings
<code>isdigit()</code> / <code>isalpha()</code>	Checks if string is all digits / all alphabetic

String Operators

PY

```
a = "Py"
b = "thon"

print(a + b)          # 'Python' - concatenation
print(a * 3)          # 'PyPyPy' - repetition

print('y' in a)       # True - membership
print('z' not in a)   # True

print(a == "Py")      # True - equality check
```

Explanation

- + (concatenation) joins two strings together into a new string
- * (repetition) repeats a string the given number of times
- in / not in check whether a substring/character exists within a string (membership operators)
- == compares the contents of two strings for equality

Joining and Combining Strings

PY

```
words = ["Python", "is", "awesome"]

sentence = " ".join(words)
print(sentence)      # 'Python is awesome'

csv_line = ",".join(words)
print(csv_line)      # 'Python,is,awesome'

full_name = "Ravi" + " " + "Kumar"
print(full_name)     # 'Ravi Kumar'
```

Explanation

- `join()` combines a list of strings into one, using the calling string as a separator
- `join()` is generally more efficient than repeated `+` concatenation in loops
- `+` can still be used for simple, one-off concatenation of a few strings

Formatting Strings

PY

```
name, marks = "Anita", 88.5

# % operator (old style)
print("%s scored %.1f" % (name, marks))

# str.format() method
print("{} scored {:.1f}".format(name, marks))

# f-string (modern, preferred)
print(f"{name} scored {marks:.1f}")
```

Explanation

- Three common ways exist to format strings in Python: % operator, .format() method, and f-strings
- f-strings (Python 3.6+) are the most readable and recommended approach
- {:.1f} inside the format specifier limits a float to one decimal place
- All three approaches above produce the same output: 'Anita scored 88.5'

Quick Check - Strings

1

What will `print("Hello"[1:4])` display?

2

Write an expression to reverse the string "Python" using slicing.

3

Which function counts how many times a character appears in a string?

4

Predict the output of: `"-".join(["2024","01","15"])`

03

Control Flow Statements

Making decisions and repeating actions in a Python program

Conditional (Decision-Making) Statements

- **if statement**
 - Executes a block only when the condition is True
- **if...else statement**
 - Chooses between two blocks based on the condition
- **if...elif...else statement**
 - Tests multiple conditions in sequence
- **Nested if**
 - An if statement placed inside another if/else block
- Python uses indentation (not braces) to define which statements belong to a block

if / elif / else - Example

PY

```
marks = 78

if marks >= 90:
    grade = "A+"
elif marks >= 75:
    grade = "A"
elif marks >= 60:
    grade = "B"
else:
    grade = "C"

print("Grade:", grade)    # Grade: A
```

Explanation

- Conditions are checked top to bottom; the first True condition's block runs and the rest are skipped
- elif lets you check several conditions without deeply nesting if statements
- else acts as a default/fallback when none of the above conditions are True
- The colon (:) and consistent indentation define each block

for Loop

PY

```
# Loop over a range of numbers
for i in range(1, 6):
    print(i, end=" ")    # 1 2 3 4 5

print()

# Loop over a string
for ch in "BCA":
    print(ch)

# Loop over a list
for subject in ["Python", "DBMS", "Stats"]:
    print(subject)
```

Explanation

- `range(start, stop)` generates numbers from start up to (not including) stop
- A for loop can iterate directly over any sequence: strings, lists, tuples, ranges
- The loop variable (`i`, `ch`, `subject`) automatically takes each value in turn

while Loop

PY

```
count = 1
while count <= 5:
    print("Count:", count)
    count += 1

# Sentinel-controlled while loop
total = 0
num = int(input("Enter a number (0 to stop): "))
while num != 0:
    total += num
    num = int(input("Enter a number (0 to stop): "))
print("Total:", total)
```

Explanation

- A while loop repeats as long as its condition remains True
- The loop variable (count) must be updated inside the loop, or it becomes an infinite loop
- A sentinel value (here, 0) is a special input that signals the loop to stop

Nested Control Flow

PY

```
for i in range(1, 4):
    for j in range(1, 4):
        if (i + j) % 2 == 0:
            print(f"({i},{j}) -> Even sum")
        else:
            print(f"({i},{j}) -> Odd sum")
```

Explanation

- A nested loop is a loop placed inside another loop - the inner loop completes fully for each iteration of the outer loop
- Conditional statements (if/else) can also be nested inside loops, and loops can be nested inside conditionals
- Indentation level clearly shows which block each statement belongs to
- Useful for tasks like generating patterns, tables and matrix-related operations

break, continue, pass and exit()

Keyword	Effect
<code>break</code>	Immediately terminates the nearest enclosing loop
<code>continue</code>	Skips the rest of the current iteration, moves to the next one
<code>pass</code>	Does nothing - a placeholder where a statement is syntactically required
<code>exit()</code> / <code>sys.exit()</code>	Terminates the entire program immediately

break and continue - Example

PY

```
# break: stop the loop early
for num in range(1, 10):
    if num == 5:
        break
    print(num, end=" ")    # 1 2 3 4

print()

# continue: skip even numbers
for num in range(1, 10):
    if num % 2 == 0:
        continue
    print(num, end=" ")    # 1 3 5 7 9
```

Explanation

- break exits the loop completely as soon as it executes - no further iterations occur
- continue skips only the current iteration and moves on to the next one - the loop keeps running
- Both are typically used together with an if condition inside a loop

pass and exit() - Example

PY

```
for i in range(5):
    if i == 2:
        pass          # placeholder, does nothing
    print(i)

import sys
marks = int(input("Enter marks: "))
if marks < 0 or marks > 100:
    print("Invalid marks entered!")
    sys.exit()        # terminate the program
print("Marks accepted:", marks)
```

Explanation

- pass is commonly used while writing code structure first and filling logic later
- Unlike continue, pass does NOT skip the rest of the loop body - it simply does nothing at that point
- sys.exit() stops the entire program - useful for validating critical input before continuing

Unit I - Complete Recap

- ✓ Python: simple, interpreted, dynamically typed; created by Guido van Rossum (1991)
- ✓ Program structure: comments, indentation, identifiers, keywords, operators, data types
- ✓ Type conversion (implicit/explicit), I/O statements: input() and print()
- ✓ Strings: creation, indexing, slicing, built-in functions, operators, joining, formatting
- ✓ Control flow: if/elif/else, for and while loops, nested control flow
- ✓ Loop control: break, continue, pass and exit() / sys.exit()

Coming up in the Lab: Lab Programs 1-9 (Prime check, Power, Even/Odd, Series, String operations, Substring search) directly apply these concepts.



Thank You

Questions & Discussion